WhatsApp Integration

Automated Incident System

Prepared by:	**Yirgalem Hailemariam**
Group:	**04 (**Automated Incident System**)**
Date:	**30/10/2025**

Introduction

Innovactions primarily uses digital communication channels to receive and manage incident reports from their clients. WhatsApp serves as the main platform for clients to report incidents and provide additional details, while Slack is used internally for collaboration and incident resolution.

To streamline this process, our goal is to redirect incident reports from WhatsApp to the company's Slack workspace, where they can be efficiently managed.

The system we are building acts as a middleware that automatically classifies the severity of incoming messages and handles bidirectional communication between platforms.

- **Inbound flow:** WhatsApp → System → Slack
- **Outbound flow:** Slack → System → WhatsApp

This integration enables automated severity classification, ticketing and status tracking of incidents.

Integration process

Since WhatsApp is owned by Meta, developers must use Meta's WhatsApp Business Platform for integration. Meta provides a sandbox environment that allows developers to experiment with and test message flows. To access this sandbox, one must register as a Meta Developer and create a WhatsApp Business App.

Once registered, developers receive a test business number and can add private numbers for testing message exchange.

WhatsApp's inbound messaging is event-driven, meaning it only delivers messages via webhooks. It does not support polling or direct API fetch requests. Therefore, webhook configuration is essential.

Webhook Configuration

In our case, the webhook is implemented in Java Spring Boot. Its responsibilities include:

- ✓ Verifying the connection between WhatsApp and the application through a handshake process (returning a challenge string).
- ✓ Listening for incoming message events and forwarding them to the system.

Webhook Connection Details:

Note: Use ngrok port forwarding for quick local testing and to expose the webhook endpoint to the internet.

- **Platform:** Meta Developers Portal
- **Endpoint URL:** https://<ngrok-generated-url>/whatsapp/webhook
- **Verify Token:** A custom string used to authenticate and verify the webhook during setup
- **Return Challenge:** The application must respond with the challenge value sent by Meta to complete verification

Messaging

The system handles two types of communication with WhatsApp: **Inbound** (messages coming from clients) and **Outbound** (messages sent back to clients).

Inbound messaging

Messages sent from a client's private WhatsApp number to the WhatsApp Business test number.

- Subscribe to the desired service (e.g., **messages**) in the Meta Developer Portal.
- When a message is sent from a registered WhatsApp number, Meta sends a JSON payload to the configured webhook endpoint.
- The payload includes metadata such as the WhatsApp Phone Number ID, message content, sender information, and timestamp.
- The webhook receives this payload for processing by the application backend.

Outbound messaging

Messages sent from the WhatsApp Business test number (through the system) to the client's private WhatsApp number. Handled using the WhatsApp Business Platform REST API, sent through Meta's Graph API.

- Base API URL: *https://graph.facebook.com/v20.0*
- Endpoint: *https://graph.facebook.com/v20.0/{phone-number-id}/messages*
- Access token: Generated in the Meta Developer Portal and used to authenticate outbound API requests.
- The *from* field represents your WhatsApp Business test number ID (sender)
- The *to* field represents the client's private number (receiver).
- Messages are sent as HTTP POST requests

Cross-Platform communication and test

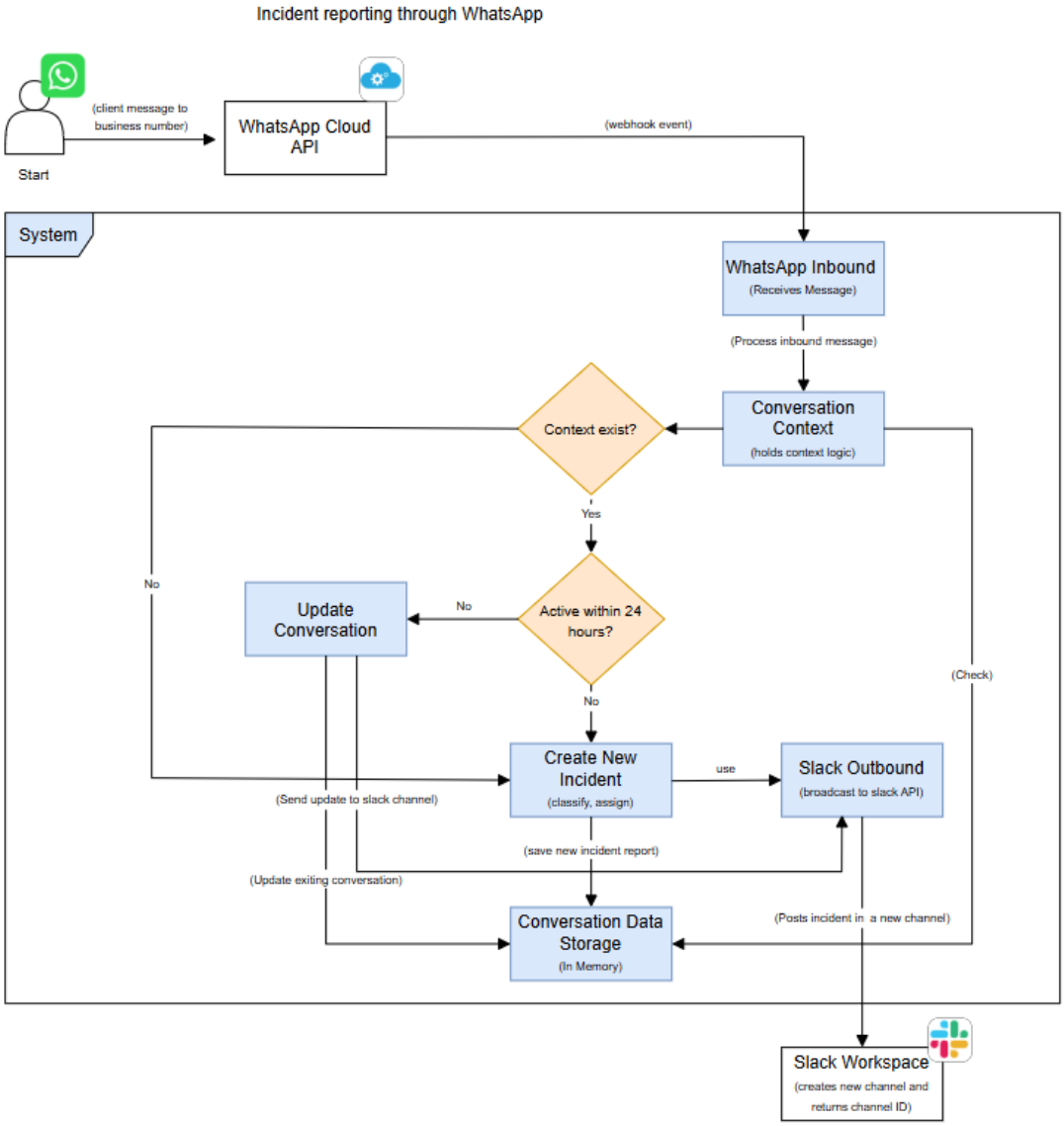

Figure 1: WhatsApp-Based Incident Reporting Flow diagram

Test Requirements

WhatsApp:

- ✓ Access to the Meta Sandbox Environment
- ✓ A registered private phone number linked to your Meta Developer account
- ✓ A Meta Business Test Number ID
- ✓ A verified webhook connection configured in the backend
- ✓ WhatsApp access token

Slack:

- ✓ A verified Slack workspace URL connection for inbound communication
- ✓ A registered URL for /close_incident endpoint to handle incident closure events through Slack

Use Case:
A client reports an incident through WhatsApp, and the system automatically creates a corresponding Slack channel with incident details.

Test Objective:
Validate end-to-end message flow from WhatsApp → System → Slack, ensuring incident creation and correct metadata display.

Test Steps:

1. From the registered private number, send a plain text message (e.g., "The dashboard is not loading") to the Meta business test number.
2. The message should trigger a webhook event that your backend processes.
3. The system classifies the message severity, creates a new incident, and broadcasts it to Slack.

Expected Outcome:

- A new Slack channel is automatically created for the incident.
- The message content and incident details (e.g., ID, timestamp, severity) are displayed.
- Reporter information (phone number) and the source platform (WhatsApp) are clearly shown in the Slack message.
- The system logs confirm that the inbound webhook event was received and processed successfully.

Use case:

A client is notified when an incident is closed, including the reason for closure.

Test Objective:

Validate the end-to-end flow from Slack → System → WhatsApp, ensuring that incident closure actions trigger notifications correctly and that the closure message includes the reason for closure.

Test Steps:

1. In Slack, trigger the incident closure event by using the /close_incident command and add a reason for closure.

2. The system receives the closure request through the registered /close_incident endpoint.
3. The system sends a notification to the client through the integrated outbound channel.

Expected Outcome:

- The incident status is updated to Closed in the system.
- The Slack channel created particularly for this incident will be removed.
- A closure confirmation message is sent to the client, including the reason for closure.
- System logs confirm that the closure event was received, processed, and the notification successfully dispatched.